

Week 2 Lab - Teaching a Machine to Learn from Data

Last time, our maze solver was AI because it made decisions by searching. But it did **not** learn.

Today we will build a tiny learning system from scratch. The system will look at experimental data from a moving cart and learn a rule:

```
predicted_position = speed * time + starting_position
```

By the end, you should be able to point to the parameters and say:

These numbers improved because of data.

We will use ordinary Python only. No NumPy, pandas, or scikit-learn.

Quick vocabulary for today

Term	Plain meaning	In our lab
data	examples the computer learns from	time and measured position pairs
feature/input	what the model sees	time t
target/output	what we want to predict	position y
model	a prediction rule	$y_{\text{hat}} = w*x + b$
parameter	a number the model can change	w and b
loss	how bad the predictions are	mean squared error
training	changing parameters to reduce loss	gradient descent
testing	checking on data not used for training	test MSE

Prediction question: If a model has lower training loss, is it automatically better on future data?

Tiny ML story before the cart

Before using physics data, look at a very small pattern:

x	y
0	1
1	3
2	5

Prediction: What should the model predict for $x = 3$?

A machine-learning model does not begin by knowing the rule. It starts with adjustable numbers and changes them so the predictions become less wrong.

```
# Tiny example: same idea as the full lab, but with very small
numbers.
toy_data = [(0, 1), (1, 3), (2, 5)]

def toy_predict(x, w, b):
    return w * x + b

# Bad starting parameters
print("bad guess for x=3:", toy_predict(3, w=1, b=0))

# Better parameters
print("better guess for x=3:", toy_predict(3, w=2, b=1))

# TODO: In one sentence, what changed between the bad guess and the
better guess?

# Run this cell first.
# The data is from a simple motion experiment: a cart moves at roughly
constant speed.
# Each pair is (time in seconds, measured position in meters).

DATA = [
    (0.0, 0.52),
    (0.5, 1.43),
    (1.0, 2.36),
    (1.5, 3.20),
    (2.0, 4.13),
    (2.5, 5.02),
    (3.0, 5.92),
    (3.5, 6.79),
    (4.0, 7.74),
    (4.5, 8.57),
]

print("number of examples:", len(DATA))
print("first example:", DATA[0])
print("last example:", DATA[-1])
```

Part 1 - Understand the data before modeling

Before building any AI model, inspect the examples.

Each pair is:

```
(time, measured_position)
```

For example, (1.0, 2.36) means:

at time 1.0 seconds, the cart was measured around 2.36 meters.

Prediction: As time increases, what should happen to position? Why?

```
# TODO 1: Print the data in a readable table.
# Desired format:
# time = 0.0 sec  position = 0.52 m
# time = 0.5 sec  position = 1.43 m
# ...

for time, position in DATA:
    # Replace pass with a print statement.
    pass
```

Part 2 - Training data vs testing data

If the model only looks good on examples it already saw, we do not know whether it learned a useful pattern.

So we split the examples:

- **TRAIN_DATA**: examples used to improve the parameters
- **TEST_DATA**: examples saved for later evaluation

This is like studying with practice problems, then checking with new problems.

```
# We will train on the first 7 examples and test on the last 3.
# This is simple and deterministic for class.

TRAIN_DATA = DATA[:7]
TEST_DATA = DATA[7:]

print("train examples:", len(TRAIN_DATA))
print("test examples:", len(TEST_DATA))
print("test data:", TEST_DATA)

# CHECKPOINT:
# Why should the model NOT use TEST_DATA while learning?
```

Part 3 - Build the prediction rule

Our model is a straight-line function:

$$y_hat = w*x + b$$

where:

- `x` is time
- `y_hat` is predicted position
- `w` is the learned speed/slope
- `b` is the learned starting position/intercept

In physics language, this is similar to:

```
position = velocity * time + initial_position

def predict(x, w, b):
    """Predict position from time using  $y_{\text{hat}} = w \cdot x + b$ ."""
    # TODO 2: return the model prediction.
    pass

# Try a model with speed = 2 and starting position = 0.
print(predict(3.0, w=2.0, b=0.0))

# Checkpoint: if  $w = 2$  and  $b = 0$ , what should  $\text{predict}(3.0)$  return?
assert predict(3.0, 2.0, 0.0) == 6.0
```

Part 4 - Measure error for one prediction

A prediction is not just right or wrong. It can be close or far.

For one example:

```
error = prediction - actual
```

If the error is positive, the model predicted too high. If the error is negative, the model predicted too low.

```
# TODO 3: Complete the one-example error calculation.

x = 2.0
actual_y = 4.13
w = 1.0
b = 0.0

prediction = predict(x, w, b)
error = None # replace None with prediction - actual_y

print("prediction:", prediction)
print("actual:", actual_y)
print("error:", error)

# Prediction question: Is this model too high or too low?
```

Part 5 - Loss: one number for how bad the model is

For the whole dataset, we need one score that tells us how bad the model is overall.

We will use **mean squared error**:

```
MSE = average((prediction - actual)^2)
```

Why square the errors?

- negative and positive errors do not cancel out
- larger mistakes are punished more

```
def mean_squared_error(data, w, b):  
    """Return the mean squared error of a model on a dataset."""  
    total = 0  
  
    # TODO 4:  
    # For each (x, actual_y) in data:  
    # 1. compute predicted_y  
    # 2. compute error = predicted_y - actual_y  
    # 3. add error**2 to total  
  
    return total / len(data)  
  
print("loss for w=1, b=0:", mean_squared_error(TRAIN_DATA, 1.0, 0.0))  
print("loss for w=2, b=0:", mean_squared_error(TRAIN_DATA, 2.0, 0.0))  
print("loss for w=2, b=0.5:", mean_squared_error(TRAIN_DATA, 2.0,  
0.5))  
  
# CHECKPOINT:  
# Which of these three guesses is best? How do you know?
```

Part 6 - Try a few guesses manually

Before we automate learning, let us act like the learning algorithm.

We will test several possible speeds w while keeping $b = 0.5$.

Prediction: Which speed will work best for this cart?

```
# TODO 5: Complete this loop to test different w values.  
  
candidate_speeds = [0.5, 1.0, 1.5, 2.0, 2.5]  
b = 0.5  
  
best_w = None  
best_loss = float("inf")
```

```

for w in candidate_speeds:
    loss = None # replace with mean_squared_error(TRAIN_DATA, w, b)
    print("w =", w, "loss =", loss)

    # TODO: update best_w and best_loss when this candidate is better.

print("best speed from candidates:", best_w)
print("best loss:", best_loss)

```

Part 7 - Gradient descent intuition

Manual guessing is slow. We want the computer to improve w and b automatically.

Gradient descent is the idea:

1. Make predictions
2. Measure loss
3. Find which direction increases loss
4. Move parameters a small step in the opposite direction
5. Repeat

For our model:

```

prediction = w*x + b
error = prediction - actual

```

The gradients for mean squared error are:

```

dw = average(2 * error * x)
db = average(2 * error)

```

You do not need to memorize these. Today, focus on what they mean:

- dw tells us how to change the slope/speed
- db tells us how to change the intercept/starting position

```

def gradients(data, w, b):
    """Return gradients (dw, db) for mean squared error."""
    total_dw = 0
    total_db = 0

    # TODO 6:
    # For each example, compute prediction and error.
    # Add 2 * error * x to total_dw.
    # Add 2 * error to total_db.

    n = len(data)

```

```
    return total_dw / n, total_db / n

print("gradients at w=0, b=0:", gradients(TRAIN_DATA, 0.0, 0.0))
```

Part 8 - The training loop

This is the moment where the model actually learns.

The parameters begin as bad guesses. Then we repeatedly update them:

```
w = w - learning_rate * dw
b = b - learning_rate * db
```

The learning rate controls the step size.

Prediction: What might happen if the learning rate is too large?

```
def train_linear_model(data, w, b, learning_rate, epochs):
    """Train w and b using gradient descent."""
    history = []

    for epoch in range(epochs):
        # TODO 7:
        # 1. compute loss
        # 2. compute dw, db
        # 3. update w and b
        # 4. append loss to history
        pass

    return w, b, history

w, b, history = train_linear_model(
    TRAIN_DATA,
    w=0.0,
    b=0.0,
    learning_rate=0.05,
    epochs=200,
)

print("learned w:", w)
print("learned b:", b)
print("first loss:", history[0])
print("final loss:", history[-1])

# CHECKPOINT:
# Did the loss decrease? Did the parameters change because of data?
```

Part 9 - Evaluate on training and testing data

Training loss tells us how well the model fits examples it learned from.

Testing loss tells us how well it works on examples it did **not** use to update parameters.

A useful model should do reasonably well on both.

```
train_loss = mean_squared_error(TRAIN_DATA, w, b)
test_loss = mean_squared_error(TEST_DATA, w, b)

print("train loss:", train_loss)
print("test loss:", test_loss)

# TODO 8: Print predictions on the test examples.
# Format: time, actual position, predicted position
for x, actual_y in TEST_DATA:
    predicted_y = None # replace with predict(x, w, b)
    print(x, actual_y, predicted_y)
```

Part 10 - Experiment with learning rates

The learning rate is not learned from the data. It is a setting chosen by the human.

Try several learning rates and compare:

- final loss
- learned w
- learned b
- whether training looks stable

Prediction: Which learning rate will be too small? Which might be too large?

```
# TODO 9: Try several learning rates.
# Suggested values: 0.001, 0.01, 0.05, 0.2

learning_rates = [0.001, 0.01, 0.05, 0.2]

for lr in learning_rates:
    w_try, b_try, hist_try = train_linear_model(
        TRAIN_DATA,
        w=0.0,
        b=0.0,
        learning_rate=lr,
        epochs=100,
    )
    print("learning rate:", lr)
```

```
print("  final loss:", hist_try[-1])
print("  w:", w_try, "b:", b_try)
```

Part 11 - Make a future prediction

Use your trained model to predict the cart position at a new time.

This is where we must be careful:

- predicting at $t = 3.2$ is interpolation: inside the range of data
- predicting at $t = 10.0$ is extrapolation: outside the range of data

Extrapolation is riskier.

```
# TODO 10: Use the trained model for new predictions.
times_to_predict = [3.2, 5.0, 10.0]

for t in times_to_predict:
    predicted_position = None # replace with predict(t, w, b)
    print("time:", t, "predicted position:", predicted_position)

# Which prediction do you trust least? Why?
```

Extension - When a straight line is not enough

Our model assumes constant speed. But what if the cart accelerates?

Then position may follow a curve, not a line.

Try this extension if you finish early:

1. Create a new dataset where position grows like $0.5 * \text{acceleration} * \text{time}^2$.
2. Train the same linear model.
3. Compare train and test loss.
4. Explain why the model fails.

This prepares us for Week 3: what happens when one simple equation is not powerful enough?